

---

# Report Summary: Pandas-I

## 1. Introduction to Pandas

### Definition

Pandas (Panel Data) is a high-level data manipulation tool used for analyzing data<sup>1</sup>. It is built on packages like NumPy and Matplotlib, providing a single, convenient place for data analysis and visualization<sup>2</sup>.+1

### Importance and Key Features

- **Simplicity:** Provides simple and intuitive syntax, making it accessible even for beginners<sup>3</sup>.
- **Efficiency:** Built on top of NumPy, it leverages efficient array operations to handle large datasets<sup>4</sup>.
- **Versatility:** Can handle various data formats including CSV, Excel, SQL, and JSON<sup>5</sup>.
- **Data Cleaning:** Offers robust tools for detecting and handling missing or inconsistent data<sup>6</sup>.

### Installation & Import

To install Pandas, Python must already be installed on the system<sup>7</sup>.

- **Command:** `conda install pandas` or `pip install pandas`<sup>888</sup>.+1
- **Import Syntax:** `import pandas as pd`<sup>9</sup>.

---

## 2. Pandas Series (Data Structure)

### Definition

A Series is a one-dimensional array-like structure that can hold a sequence of values of any data type (integers, floats, strings, etc.)<sup>10</sup>. Each value is associated with a unique label known as the index<sup>11</sup>.

+1

### Comparison: Pandas Series vs. NumPy Arrays

| Feature   | Pandas Series                                                         | NumPy Arrays                                                                       |
|-----------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Indexing  | Supports custom labeled indices (numbers or letters) <sup>12</sup> .  | Accessed by integer position only <sup>13</sup> .                                  |
| Alignment | Generates NaN (missing values) if data is not aligned <sup>14</sup> . | Alignment fails if there are no matching values; no concept of NaN <sup>15</sup> . |

| Feature | Pandas Series                        | NumPy Arrays                         |
|---------|--------------------------------------|--------------------------------------|
| Memory  | Requires more memory <sup>16</sup> . | Occupies less memory <sup>17</sup> . |

## Creating a Series

A Series can be created using `pd.Series(data, index=...)`.

1. **From Scalar Values:** A single value is repeated for each index label<sup>18181818</sup>.

+1

Python

```
s = pd.Series(10, index=['a', 'b', 'c'])
```

2. **From a List:** Default numeric index (0, 1, 2...) is assigned if not specified<sup>19191919</sup>.

+1

Python

```
data = [10, 20, 30]
```

```
s = pd.Series(data) # or specify index=['a', 'b', 'c']
```

**Note:** If the length of the index list does not match the data elements, a `ValueError` is raised<sup>20202020</sup>.

+1

3. **From NumPy Array:** Useful for numerical operations<sup>21</sup>.

Python

```
arr = np.array([10, 20])
```

```
s = pd.Series(arr)
```

4. **From Dictionary:** Keys automatically become the index labels<sup>22222222</sup>.

+1

Python

```
d = {'a': 10, 'b': 20}
```

```
s = pd.Series(d)
```

## Accessing Elements

- **Indexing:** Similar to NumPy. Supports both positional index (integer starting from 0) and labeled index (user-defined)<sup>23232323</sup>.

+1

- **Slicing:**

- **Positional Slicing:** `s[start:end]` — The end index is **exclusive**<sup>24</sup>.

- **Label Slicing:** `s['start':'end']` — The end label is **inclusive**<sup>25</sup>.

## Key Attributes of Series

- `series.index`: Returns the labels of the Series<sup>26</sup>.
- `series.values`: Returns the values of the Series<sup>27</sup>.
- `series.name`: Returns or sets the name of the Series<sup>28</sup>.
- `series.dtype`: Returns the data type of the values<sup>29</sup>.
- `series.shape`: Returns the dimensions (tuple)<sup>30</sup>.
- `series.size`: Returns the total number of elements<sup>31</sup>.
- `series.empty`: Returns True if the Series is empty<sup>32</sup>.
- `series.hasnans`: Returns True if the Series contains NaN values<sup>33</sup>.
- `series.is_unique`: Returns True if all elements are unique<sup>34</sup>.

## Key Methods of Series

- `.head(n)` / `.tail(n)`: Returns the first/last `n` elements (default is 5)<sup>35353535</sup>.

+1

- `.describe()`: Generates descriptive statistics (count, mean, std, min, max, etc.)<sup>36</sup>.
- `.value_counts()`: Returns counts of unique values<sup>37</sup>.
- `.sort_values()`: Sorts by values (default ascending; use `ascending=False` for descending)<sup>38</sup>.
- `.sort_index()`: Sorts by index labels<sup>39</sup>.
- `.drop(label)`: Removes specified index labels. Returns a new Series unless `inplace=True` is used<sup>40404040</sup>.
- +1
- `.replace(old, new)`: Replaces specified values<sup>41</sup>.
- `.isnull()` / `.notnull()`: Detects missing/non-missing values<sup>42424242</sup>.
- +1

## Mathematical Operations

Operations on Series are element-wise and align based on the index<sup>43434343</sup>.

+1

- **Index Matching:** If an index exists in one Series but not the other, the result is NaN<sup>44</sup>.
- **Handling NaN:**
  - `series.fillna(value)`: Fills NaN with a specific value (e.g., 0)<sup>45</sup>.
  - `series.dropna()`: Removes NaN values<sup>46</sup>.

---

## 3. Pandas DataFrame (Data Structure)

### Definition

A DataFrame is a two-dimensional labelled data structure, similar to a SQL table or Excel sheet<sup>47</sup>. It contains rows and columns, allowing for heterogeneous data types (numeric, string, boolean) across different columns<sup>48</sup>.

+1

### Creating a DataFrame

1. **Empty DataFrame:** `pd.DataFrame()`<sup>49</sup>.
2. **From NumPy Array:** Requires specifying column names<sup>50</sup>.  
Python  
`df = pd.DataFrame(np_array, columns=['Age', 'City'])`
3. **From List of Dictionaries:** Each dictionary represents a row; keys become column headers<sup>51</sup>.  
Python  
`data = [{'Name': 'Alice', 'Age': 25}, {'Name': 'Bob', 'Age': 30}]`  
`df = pd.DataFrame(data)`
4. **From Files:**
  - **Excel:** `pd.read_excel("path/file.xlsx", sheet_name="Sheet1")`<sup>52525252</sup>.
  - +1
  - **CSV:** `pd.read_csv("path/file.csv")`<sup>53</sup>.

### Accessing Data

#### 1. Accessing Columns

- **Single Column:** `df['column_name']` returns a Series<sup>54</sup>.
- **Multiple Columns:** `df[['col1', 'col2']]` returns a new DataFrame<sup>55</sup>.

#### 2. Accessing Rows (Label vs. Position)

- **.loc (Label-based):** Accesses rows/columns by their names<sup>56</sup>.
  - **Syntax:** `df.loc[row_label, col_label]`<sup>57</sup>.

- *Slicing*: The end point is **inclusive**<sup>58</sup>.
- **.iloc (Position-based)**: Accesses rows/columns by integer position (0-based)<sup>59</sup>.
  - *Syntax*: `df.iloc[row_index, col_index]`<sup>60</sup>.
  - *Slicing*: The end point is **exclusive**<sup>61</sup>.

### 3. Boolean Indexing

Used to filter rows based on a condition<sup>62</sup>.

- *Example*: `df[df['gender'] == 'Male']` selects all rows where gender is Male<sup>63</sup>.
- *Complex Filters*: Use logical operators & (and), | (or), ~ (not).
  - *Example*: `df[(df['name'] == 'Tobit') | (df['name'] == 'Natale')]`<sup>64</sup>  
+1

### 4. Slicing

- **Using .loc**: `df.loc[2:4, ['col1', 'col2']]` slices rows 2 through 4 (inclusive) and specific columns<sup>65</sup>  
+1
- **Using .iloc**: `df.iloc[1:4, 1:3]` slices rows 1 to 3 (exclusive) and columns 1 to 2 (exclusive)<sup>66</sup>  
+1